

GABRIEL POPA

Senior Software Engineer - Angular • Node (ExpressJS/ NestJS) • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | <https://www.linkedin.com/in/gabriel-popa-12890b414/>

ABOUT ME

Senior Software Engineer with **11+ years** of experience building scalable **Angular** frontends and reliable **Node.js** backend services for modern web platforms, turning changing business requirements into stable releases and maintainable systems. Strong specialization in **Angular 8–18**, **TypeScript 3.x–5.x**, **Node.js 12–20**, **NestJS**, **Express.js**, **REST APIs**, **PostgreSQL**, **Redis**, **Kafka**, **RabbitMQ**, and **AWS**, with proven success modernizing legacy modules, fixing difficult production issues, delivering secure integrations, implementing new platform features, and guiding safe migrations from older UI and API patterns to cleaner, faster, and more observable architectures.

SKILLS

- **Frontend: Angular 8–18**, TypeScript 3.x–5.x, JavaScript ES6+, RxJS, NgRx, Angular CLI, Angular Material, Reactive Forms, Template-Driven Forms, Route Guards, Lazy Loading, reusable component architecture, SPA development, HTML5, CSS3, SCSS, responsive UI, Webpack, Vite basics, state management, client-side validation
- **Backend & APIs: Node.js 12–20**, **Express.js**, **NestJS 7–10**, REST APIs, GraphQL basics, WebSockets, JWT authentication, OAuth2, RBAC, middleware architecture, background workers, job queues, API versioning, file processing, event-driven services, OpenAPI/Swagger
- **Data & Messaging:** PostgreSQL, MySQL, Redis, Kafka, RabbitMQ, Elasticsearch, OpenSearch basics, query optimization, indexing, schema design, caching strategies, event processing, pub/sub messaging
- **Cloud & DevOps:** AWS, EC2, S3, RDS, ECS, EKS basics, IAM basics, Docker, GitHub Actions, GitLab CI, CI/CD pipelines, Nginx, Linux, containerized deployments, environment configuration, build pipelines, release automation
- **Observability / Quality / Security:** Sentry, structured logging, OpenTelemetry basics, Prometheus/Grafana basics, correlation IDs, rate limiting, OWASP security practices, Jest, Supertest, Cypress, Playwright, unit testing, integration testing, contract testing, end-to-end testing, error monitoring

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania (Jan 2024 – May 2026)

- Led delivery of a multi-tenant operational platform using **Angular14-18**, **TypeScript**, and **Node.js**, translating fragmented dashboard, reporting, and administration requirements into stable workflows with reusable modules and cleaner ownership boundaries.
- Modernized legacy frontend areas by restructuring large feature screens into **Angular** feature modules, shared components, route guards, and strongly typed services, which reduced regressions and made future enhancements easier to deliver.
- Improved rendering speed on data-heavy pages by profiling change detection pressure, introducing smarter observable flows with **RxJS**, and reducing unnecessary template recomputation that had been causing UI lag during filtering.
- Migrated legacy **NgModule-based Angular** screens to a **standalone/component-driven architecture** across **Angular 14–18**, improving reusability, reducing coupling, and lowering future upgrade complexity.
- Implemented **dynamic form workflows** with **Reactive Forms**, custom validators, and reusable error display patterns, which improved validation consistency and reduced repeated defects across onboarding and administration screens by **33%**.
- Stabilized a recurring issue where real-time tenant status panels stopped updating after temporary disconnects by improving **WebSocket** reconnection handling and buffering missed events before UI refresh cycles resumed.
- Built modular backend services with **Node.js**, **NestJS**, and **TypeScript**, creating maintainable **REST** endpoints with clearer controller and service separation that improved testing, onboarding, and change safety.

- Handled a safe migration of a legacy API runtime from older **Node.js** patterns to a cleaner **Node.js 20** deployment model through staged dependency updates, compatibility checks, and rollout controls that avoided disruption.
- Resolved a production memory issue caused by stale listeners and unmanaged in-memory caching by using heap inspection, log correlation, and load validation, restoring stable behavior during sustained traffic periods.
- Prevented duplicate billing and action processing by implementing idempotent write handling, refining retry behavior, and tightening **PostgreSQL** constraints where asynchronous retries had previously caused inconsistent records.
- Strengthened platform security by implementing **JWT**, **OAuth2**, role-aware UI rendering, request throttling, and audit logging so that sensitive administration functions were easier to control and investigate.
- Integrated **Kafka** and **RabbitMQ** for asynchronous event flows covering notifications, export generation, and downstream status updates, reducing tight coupling between user-triggered actions and background processing by **38%**.
- Improved incident visibility by adding structured logs, **Sentry**, and **OpenTelemetry**-based telemetry around critical API and queue paths, making it easier to isolate tenant-specific failures before they spread broadly.

Software Engineer

Next Apps | Ghent, Belgium (*Nov2020 – Nov 2023*)

- Built customer-facing product workflows with **Angular** and **TypeScript**, focusing on validation-heavy pages, reusable UI behavior, and clearer state transitions for screens that handled complex user submissions.
- Created a shared component library using **Angular Material**, reusable input wrappers, tables, modals, and feedback patterns, which reduced duplication and made product-wide UI changes significantly easier to apply.
- **Improved maintainability** by replacing scattered template logic with cleaner **RxJS** pipelines and service-layer state handling, reducing side-effect bugs by **31%** and making data flow easier to reason about.
- Migrated older form implementations to **Angular 9–16 Reactive Forms**, centralizing validation rules and improving consistency across large submission flows while reducing front-end defects caused by duplicated validation logic.
- Simplified build and release workflows by cleaning up heavy **Webpack** configuration, reducing unstable plugin behavior, and making local setup and CI execution more predictable for the team.
- Modernized a legacy **Express.js** application by migrating selected modules to **NestJS**, introducing modular controllers, dependency injection, and clearer validation structure without forcing a risky full rewrite.
- Developed versioned REST endpoints in **Node.js** with safer deprecation paths so downstream clients could move gradually instead of breaking whenever shared response contracts changed.
- Implemented **Redis caching** for hot endpoints using explicit invalidation and **TTL** strategies, reducing database pressure by **37%** while preserving data freshness for commonly accessed screens.
- Resolved intermittent **502/504** issues by aligning **Nginx** timeout behavior with backend processing paths, optimizing slow database queries, and adding latency visibility around the most sensitive user journeys.
- Fixed a hard-to-diagnose consistency bug caused by out-of-order async events by enforcing ordering keys and building a small reconciliation worker that corrected records after failure scenarios.
- Improved job reliability by adding retry policies, dead-letter handling, and operational dashboards for async workers, making failed background processes easier to detect, replay, and support.

Software Developer

Datamart | Stockholm, Sweden (*Feb 2016 – Sep 2020*)

- Built internal analytics and administration tools using early **Angular** and **Node.js** patterns, helping teams work with large datasets through responsive views, server-side pagination, and practical filtering workflows.
- Supported a gradual backend refactor from tightly coupled route files into smaller service-oriented modules, which improved maintainability while preserving behavior relied on by older internal integrations.
- Optimized **report performance** by diagnosing slow joins, adding missing **PostgreSQL** indexes, reviewing execution plans, and reducing heavy query paths, cutting export and dashboard query latency by **43%** and improving responsiveness for end users.
- Modernized older UI screens by moving brittle controller-heavy frontend logic into more maintainable **Angular** component patterns, which made change tracking and reuse easier across repeated admin workflows.
- Resolved a difficult reporting issue caused by timezone mismatches between browser input, API serialization, and database storage by standardizing date handling rules across the full request lifecycle.
- Implemented background processing for large exports and long-running report generation, replacing synchronous request handling that regularly timed out and giving users clearer progress visibility.

- Added automated integration coverage around sensitive business flows so that common regression paths were detected earlier during releases instead of surfacing only after production deployment.
- Improved CI reliability by separating fast verification steps from heavier integration execution, which reduced noisy pipeline failures and gave developers quicker feedback on safe code changes.
- Introduced audit logging for record updates and operational approvals, helping support teams trace user actions more clearly when data disputes or unexpected changes were reported.
- Reduced frontend asset weight through code splitting, dependency cleanup, and better loading boundaries, which improved page startup time for users on slower machines and networks.

Junior Software Developer

Berg Software | Timișoara, Romania (Sep2014 – Mar 2016)

- Built early web modules using **Node.js**, **Express.js**, JavaScript, and server-rendered UI patterns, helping deliver consistent request handling and basic API responses for internal and client-facing features.
- Improved **legacy web pages** by introducing modular **JavaScript**, clearer validation feedback, and better form behavior, reducing user input mistakes and repeated support follow-up by **27%**.
- Helped fix a recurring logout issue by correcting session cookie configuration and aligning backend timeout handling with browser behavior, improving login stability without forcing risky architectural change.
- Developed reusable frontend helpers and shared page components that reduced copy-paste implementation, making small feature requests and visual fixes easier for the team to deliver.
- Improved page speed by enabling compression, adjusting browser caching behavior, and removing blocking scripts that were delaying first render on frequently used screens.
- Supported release work by writing safer deployment scripts and maintaining setup documentation, which reduced environment mistakes and helped the team deliver updates more consistently.
- Assisted with bug resolution by reviewing logs, reproducing issues locally, and applying targeted low-risk fixes under senior guidance instead of making broad changes in fragile areas.
- Improved admin dashboard query performance with safer SQL patterns and basic indexing, helping the application handle growing record volume more reliably during every day operational use.
- Used structured debugging practices with reproduction notes, log review, and incremental patches, which improved fix quality and reduced the chance of introducing new bugs during urgent issue handling.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERTIFICATIONS

AWS Certified Developer – Associate — 2024

- Demonstrated practical capability in building, deploying, and maintaining **Node.js-based backend services** on **AWS**, with focus on service integration, monitoring, secure configuration, and production-ready cloud operations.

OpenJS Node.js Application Developer (JSNAD) — 2023

- Validated hands-on understanding of **Node.js** application development, including asynchronous programming, package management, debugging, testing, and building maintainable backend services with modern JavaScript patterns.

PostgreSQL for Developers — 2022

- Demonstrated working knowledge of relational modeling, query optimization, indexing strategies, transaction-aware data access, and performance-focused backend persistence patterns commonly used in **API-driven Node.js platforms**.